

Ticker Performance

Declare variables

```
ticker_str = "IBIT"  
set.seed(123)  
train_frac <- 0.6  
start_date <- "2025-01-01"  
end_date <- Sys.Date()
```

Generate returns and metrics

```
# 1) Download data: ticker (price) and VIX (implied vol)
```

```
getSymbols(c(ticker_str, "^VIX"), src="yahoo", from=start_date, to=end_date, a  
uto.assign=TRUE)
```

```
## Warning: ^VIX contains missing values. Some functions will not work if obj  
ects
```

```
## contain missing values in the middle of the series. Consider using na.omit  
(  
,
```

```
## na.approx(), na.fill(), etc to remove or replace them.
```

```
## [1] "IBIT" "VIX"
```

```
# Adjusted close for returns
```

```
ticker <- na.omit(Cl(get(ticker_str)))  
vix <- na.omit(Cl(`VIX`))
```

```
# 2) Compute basic returns & target
```

```
ret <- dailyReturn(ticker, type = "log")  
target <- lead(ret, 1) # next-day return  
direction <- ifelse(target > 0, 1, 0)
```

```
# Convert to xts explicitly
```

```
target_xts <- xts(target, order.by = index(ret))  
direction_xts <- xts(direction, order.by = index(ret))
```

```
# ---- 3) Volatility features ----
```

```
# Rolling realized volatility (21-day)
```

```
roll_sd <- rollapply(as.numeric(ret), width = 21,  
FUN = sd, na.rm = TRUE, align = "right", fill = NA)  
roll_sd_xts <- xts(roll_sd, order.by = index(ret))
```

```
# GARCH(1,1) conditional volatility
```

```
spec <- ugarchspec(variance.model = list(model="sGARCH", garchOrder=c(1,1)),
```

```

        mean.model = list(armaOrder=c(0,0)))
garch_fit <- try(ugarchfit(spec, data = coredata(na.omit(ret))), silent=TRUE)

if(inherits(garch_fit, "try-error")) {
  garch_sigma_xts <- xts(rep(NA, NROW(ret)), order.by=index(ret))
} else {
  garch_sigma <- sigma(garch_fit)
  garch_sigma_xts <- xts(rep(NA, NROW(ret)), order.by=index(ret))
  garch_sigma_xts[index(ret)[(NROW(ret)-length(garch_sigma)+1):NROW(ret)]] <-
garch_sigma
}

# Align VIX
vix_xts <- merge(ret, vix, all = FALSE)[,2]

# ---- 4) Momentum features ----
mom5 <- rollapply(as.numeric(ticker), width = 5,
  FUN = function(x) (last(x)/first(x) - 1),
  align = "right", fill = NA)
mom5_xts <- xts(mom5, order.by = index(ticker))

# 21-day momentum
mom21 <- rollapply(as.numeric(ticker), width = 21,
  FUN = function(x) (last(x)/first(x) - 1),
  align = "right", fill = NA)
mom21_xts <- xts(mom21, order.by = index(ticker))

# ---- 5) Lagged returns ----
lag1_xts <- lag(ret, 1)
lag2_xts <- lag(ret, 2)

# ---- 6) Merge all features ----
df <- merge(ret, target_xts, direction_xts, roll_sd_xts, garch_sigma_xts,
  vix_xts, mom5_xts, mom21_xts, lag1_xts, lag2_xts)

names(df) <- c("ret", "target", "direction", "roll_sd", "garch_sigma", "vix",
  "mom5", "mom21", "lag1", "lag2")

df <- na.omit(df) # drop rows with NA

# ---- 7) Train/test split ----

n <- nrow(df)
train_n <- floor(n * train_frac)
train <- df[1:train_n,]
test <- df[(train_n+1):n,]

features <- c("roll_sd", "garch_sigma", "vix", "mom5", "mom21", "lag1", "lag2")

```

```

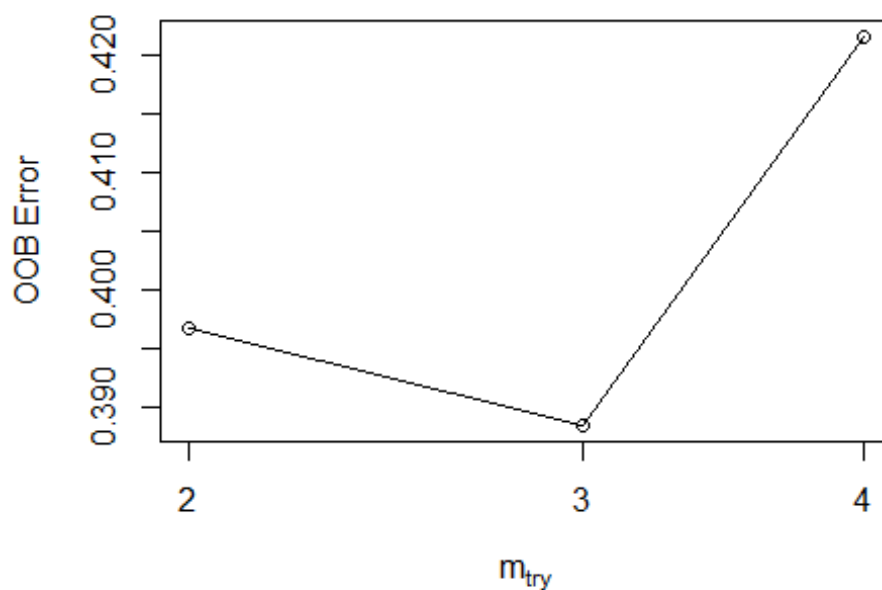
train_x <- as.data.frame(coredata(train[,features]))
train_y <- as.factor(coredata(train$direction))
test_x  <- as.data.frame(coredata(test[,features]))
test_y  <- as.factor(coredata(test$direction))

# ---- 8) Random Forest Classification ----
features_new <- c("roll_sd", "garch_sigma", "vix", "mom5", "mom21", "lag1", "lag2")
train_x_new <- as.data.frame(coredata(train[, features_new]))
test_x_new  <- as.data.frame(coredata(test[, features_new]))

best.mtry <- tuneRF(
  x = train_x_new,
  y = train_y,
  stepFactor = 1.5,
  improve = 0.01,
  ntreeTry = 200,
  trace = TRUE,
  plot = TRUE,
  doBest = TRUE)

## mtry = 2   OOB error = 39.67%
## Searching left ...
## Searching right ...
## mtry = 3   OOB error = 38.84%
## 0.02083333 0.01
## mtry = 4   OOB error = 42.15%
## -0.08510638 0.01

```



```

mtries = best.mtry$mtry

res <- tuneNTrees(
  train_x_new, train_y,
  ntreeStart = 200,
  stepFactor = 1.5,
  improve = 0.5/100, # require 0.5% OOB improvement
  maxTrees = 2000,
  mtry = mtries
)

## ntree=200 OOB=0.404959
## ntree=300 OOB=0.454545 Δrel=-12.245%

ntrees = res$best.ntree

rf_model <- randomForest(x=train_x_new, y=train_y, ntree=ntrees, mtry = mtries, importance=TRUE)

# ---- 9) Predictions ----
pred_prob <- predict(rf_model, newdata=test_x_new, type="prob")[,2]
pred_class <- ifelse(pred_prob > 0.5, 1, 0)

confusion <- table(Predicted = pred_class, Actual = as.numeric(as.character(test_y)))
accuracy <- sum(diag(confusion))/sum(confusion)
cat("Direction prediction accuracy:", round(accuracy,4), "\n")

## Direction prediction accuracy: 0.5488

print(confusion)

##           Actual
## Predicted  0  1
##           0 34 26
##           1 11 11

# ---- 10) Backtest simple strategy ----
rets_test <- as.numeric(test$target)
strategy_signal <- pred_class
strat_rets <- strategy_signal * rets_test

trade_cost_bps <- 0.0005
trade_changes <- c(0, abs(diff(strategy_signal)))
costs <- trade_changes * trade_cost_bps
strat_rets_net <- strat_rets - costs

```

```

cat("Strategy annualized return (net):",
    round(Return.annualized(xts(strat_rets_net, index(test)))*100,2), "%\n")

## Strategy annualized return (net): -20.44 %

cat("Buy & Hold annualized return:",
    round(Return.annualized(xts(rets_test, index(test)))*100,2), "%\n")

## Buy & Hold annualized return: -66.44 %

cat("Strategy Sharpe ratio:",
    round(SharpeRatio.annualized(xts(strat_rets_net, index(test))),3), "%\n")

## Strategy Sharpe ratio: -1.136

rf_reg <- randomForest(x=train_x, y=coredata(train$target), ntree=ntrees)

## Warning in rfout$mse/(var(y) * (n - 1)/n): Recycling array of length 1 in
vector-array arithmetic is deprecated.
## Use c() or as.vector() instead.

pred_return <- predict(rf_reg, newdata=test_x)
rmse <- sqrt(mean((pred_return - coredata(test$target))^2))
cat("RMSE for return prediction:", round(rmse, 6), "%\n")

## RMSE for return prediction: 0.024717

forward_pred <- data.frame(
  Date = index(test),
  ProbUp = pred_prob,
  Signal = pred_class,
  ActualDirection = as.numeric(as.character(test_y))
)

head(forward_pred)

##           Date ProbUp Signal ActualDirection
## 1 2025-07-29  0.475      0                0
## 2 2025-07-30  0.435      0                0
## 3 2025-07-31  0.505      1                0
## 4 2025-08-01  0.570      1                1
## 5 2025-08-04  0.370      0                0
## 6 2025-08-05  0.360      0                1

forward_pred$ActualNum <- ifelse(forward_pred$ActualDirection == "Up" |
                                forward_pred$ActualDirection == 1, 1, 0)

```

Visuals

```

pp = ggplot(forward_pred, aes(x=Date)) +
  geom_line(aes(y=ProbUp), color="blue") +
  geom_point(aes(y=ActualNum), color="red", size=2) +
  labs(title="Predicted Probability of Up Day",

```

```

    y="Probability (blue) / Actual Direction (red)", x="Date") +
    theme_minimal()

importance_data = importance(rf_model)
importance_df <- as.data.frame(importance_data)
importance_df$feature <- rownames(importance_df)

vi = ggplot(importance_df, aes(x = reorder(feature, MeanDecreaseGini), y = MeanDecreaseGini)) +
  geom_bar(stat = 'identity', fill = "steelblue") +
  coord_flip() + # Flips the coordinates for horizontal bars
  theme_minimal() +
  labs(title = "Variable Importance Plot",
       x = "Feature",
       y = "Mean Decrease Gini")

rets <- xts(cbind(strat_rets_net, rets_test), order.by = index(test))
colnames(rets) <- c("ActiveTrade", "BuyHold")

cumrets <- cumprod(1 + rets) - 1

# Convert to tidy data frame for ggplot
cumrets_df <- fortify.zoo(cumrets) %>%
  pivot_longer(cols = -Index, names_to = "Strategy", values_to = "CumulativeReturn")

# Plot with ggplot
gg_per <- ggplot(cumrets_df, aes(x = Index, y = CumulativeReturn, color = Strategy)) +
  geom_line(size = 1) +
  scale_y_continuous(labels = scales::percent_format()) +
  labs(
    title = "Active Trading vs Buy-and-Hold",
    x = "Date",
    y = "Cumulative Return",
    color = ""
  ) +
  theme_minimal(base_size = 14) +
  theme(legend.position = "bottom")

## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use `linewidth` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.

# Collage of charts
collage <- ggarrange(
  pp,
  vi,

```

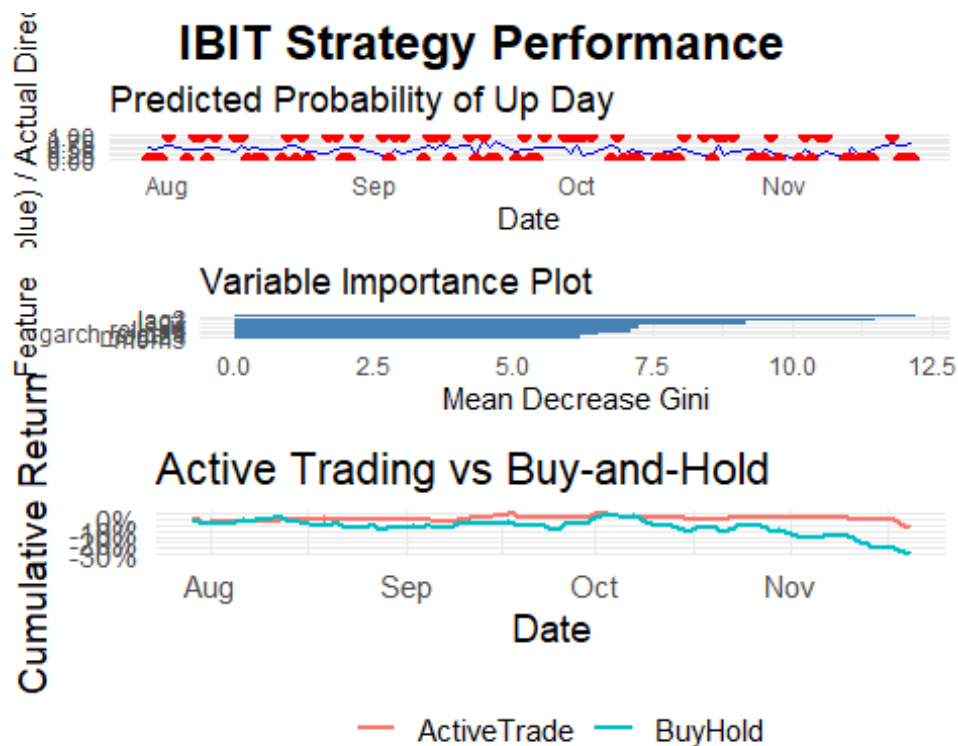
```

gg_per,
ncol = 1,
heights = c(1, 1, 2)
)

# Add a global title
collage_titled <- annotate_figure(
  collage,
  top = text_grob(
    paste(ticker_str, "Strategy Performance"),
    face = "bold",
    size = 18,
    color = "black"
  )
)

# Display
print(collage_titled)

```



```

# Optional: print your accuracy metric after
cat("Direction prediction accuracy:", round(accuracy, 4), "\n")

## Direction prediction accuracy: 0.5488

```